
thePower
FP Oficial

CONTENTIO

SaaS de Gestión de Activos Digitales (DAM)

Ciclo Formativo de Grado Superior en Desarrollo de Aplicaciones Multiplataforma (DAM)
Módulo: Proyecto Intermodular (TFG)

Autor:

Oscar Felipe Vargas García

Centro:

Prometeo The Power FP Oficial

Modalidad:

Online

Tutora:

Olga M. Moreno Martín

Curso académico:

2025 / 2026

Fecha de entrega del anteproyecto:

22/02/2026

Nombre del proyecto:

Contentio

Tipo de proyecto:

Plataforma web SaaS de Gestión de Activos Digitales (DAM)

Descripción breve:

Contentio es una plataforma web orientada a la gestión, planificación, revisión y aprobación de contenidos digitales entre una agencia y clínicas estéticas, permitiendo centralizar activos multimedia, gestionar flujos de aprobación, registrar comentarios, mantener un histórico de versiones y asegurar la trazabilidad de todo el proceso de creación de contenido.

Contexto de uso:

Relación profesional entre agencia creativa y clínicas estéticas para la gestión de contenidos destinados a redes sociales y comunicación digital.

Tecnologías principales:

- Backend: -PHP 8.3, Laravel 11, Eloquent ORM, Laravel Sanctum (autenticación API para SPA), Laravel Storage (sistema de archivos, preparado para S3)

- Frontend: Vue.js 3 (Composition API), Vue Router (navegación SPA), Pinia (gestión de estado), Axios (comunicación con la API) , Vite (bundler), Tailwind CSS

- Base de datos: PostgreSQL

- Infraestructura: Docker, Docker Compose, Nginx (sirve la SPA y reverse proxy a la API), PHP-FPM, Node.js (solo para build del frontend)

- Control de versiones: Git

- Almacenamiento: Local en fase MVP, preparado para Object Storage (S3 compatible) gracias a Laravel Storage

Alcance del MVP:

- Gestión de usuarios y roles (agencia y cliente)
- Gestión de clínicas
- Creación y planificación de contenidos
- Sistema de comentarios y revisión
- Flujo de estados (borrador, en revisión, aprobado, rechazado)
- Historial de versiones y cambios de estado
- Subida y gestión de archivos multimedia
- Trazabilidad de acciones

Despliegue previsto:

VPS en OVH utilizando Docker Compose con Nginx, PHP-FPM y PostgreSQL.

Líneas de evolución futura:

- Migración completa a Object Storage
- Integraciones con redes sociales

- Sistema de métricas y reporting

- Desarrollo de aplicación móvil

- Despliegue automatizado con Coolify

2.Definición del proyecto

El proyecto Contentio nace a partir de la observación directa de los problemas que existen hoy en la gestión de contenidos digitales dentro del ámbito de la creación audiovisual para clínicas estéticas. En concreto, su origen está en la experiencia profesional de una creadora de contenido audiovisual que trabaja de forma autónoma con este tipo de clientes, lo que ha permitido ver de primera mano las dificultades reales en la comunicación, la revisión y la aprobación de materiales.

En este contexto, es habitual que creadores de contenido y pequeñas agencias trabajen con varios clientes al mismo tiempo sin disponer de una herramienta específica que centralice todo el proceso. Aunque existen plataformas de gestión de activos digitales y de trabajo colaborativo, la mayoría están pensadas para grandes empresas, tienen un coste elevado y resultan demasiado complejas para profesionales independientes o agencias pequeñas. Además, suelen ser herramientas genéricas, poco adaptadas a la forma real de trabajar en la creación de contenido para redes sociales.

Por este motivo, muchos profesionales terminan utilizando una mezcla de herramientas no especializadas, como servicios de almacenamiento en la nube, aplicaciones de mensajería o plataformas de organización generalistas. Esta forma de trabajar, repartida en varias herramientas, suele provocar pérdida de tiempo, problemas de coordinación, falta de un seguimiento claro de los cambios y dificultades para mantener un histórico ordenado de versiones, revisiones y aprobaciones.

A todo esto se suma la naturaleza efímera de ciertos formatos, como las historias en redes sociales, que desaparecen a las 24 horas de su publicación. Esto hace que una parte importante del trabajo realizado no quede registrada de forma permanente, lo que puede dar una imagen incompleta del volumen real de trabajo que se entrega al cliente.

Contentio se plantea como una plataforma web tipo SaaS de gestión de activos digitales (DAM) pensada específicamente para este tipo de profesionales. La aplicación permitirá centralizar la planificación de contenidos, la gestión de archivos multimedia, los procesos de revisión y aprobación, los comentarios y sugerencias, así como el histórico de versiones y cambios de estado, asegurando que todo el trabajo quede correctamente registrado y sea fácilmente consultable.

De este modo, el proyecto busca ofrecer una solución más sencilla, accesible y adaptada a las necesidades reales de creadores de contenido y pequeñas agencias, mejorando tanto la organización del trabajo como la transparencia en la relación con sus clientes.

2.1Project Definition

The Contentio project was born from the direct observation of the problems that currently exist in the management of digital content within the field of audiovisual production for aesthetic clinics. More specifically, its origin is based on the professional experience of a freelance audiovisual content creator working with this type of client, which made it possible to identify first-hand the real difficulties in communication, content review and approval processes.

In this context, it is common for content creators and small agencies to work with several clients at the same time without having a specific tool that centralises the whole process. Although there are digital asset management platforms and collaborative tools, most of them are designed for large companies, have high costs and are too complex for freelancers or small agencies. In addition, they usually offer generic solutions that do not adapt well to the real workflow of social media content creation.

For this reason, many professionals end up using a combination of non-specialised tools, such as cloud storage services, messaging applications or general-purpose organisation platforms. This way of working, spread across several tools, often leads to time loss, coordination problems, lack of proper tracking of changes and difficulties in keeping a clear and organised history of versions, reviews and approvals.

On top of that, the ephemeral nature of certain formats, such as social media stories that disappear after 24 hours, means that an important part of the work produced is not permanently recorded. This can give an incomplete picture of the actual amount of work delivered to the client.

Contentio is proposed as a SaaS web platform for Digital Asset Management (DAM) designed specifically for this type of professional. The application will allow content planning, multimedia file management, review and approval processes, comments and suggestions, as well as version history and state changes to be managed in a single place, ensuring that all work is properly recorded and easy to consult.

In this way, the project aims to provide a simpler, more accessible and more suitable solution for content creators and small agencies, improving both work organisation and transparency in the relationship with their clients.

3. Objetivos del proyecto

3.1 Objetivo general

Desarrollar una plataforma web tipo SaaS para la gestión de contenidos digitales, pensada para creadores de contenido y pequeñas agencias, que permita organizar, revisar y aprobar materiales multimedia con sus clientes, dejando constancia de todo el proceso, y que pueda desplegarse en un VPS de OVH Cloud utilizando Laravel y PHP como base del backend y Vue. js como frontend.

3.2 Objetivos específicos

1. Crear un sistema de usuarios con distintos roles (agencia, creador y cliente), de forma que cada uno solo pueda acceder a las funciones que le corresponden.

2. Desarrollar un espacio donde se puedan gestionar las clínicas y sus contenidos, organizando los materiales por cliente y por tipo de publicación.

3. Implementar la subida y gestión de archivos multimedia, como imágenes y vídeos, guardándolos inicialmente en el servidor y dejando preparada la aplicación para usar almacenamiento externo en el futuro.

4. Crear un flujo de revisión de contenidos que permita cambiar el estado de cada elemento (borrador, en revisión, aprobado o rechazado) y que guarde un historial de esos cambios.

5. Desarrollar un sistema de comentarios que permita al cliente y al creador comunicarse sobre cada contenido, incluyendo la posibilidad de marcar puntos concretos de mejora sobre el material, de forma similar a herramientas como Figma, pero de manera sencilla y práctica.

6. Añadir la opción de definir un plazo máximo para la aprobación de un contenido, de forma que, si el cliente no realiza ninguna acción dentro del tiempo establecido, el contenido pueda considerarse aprobado automáticamente.

7. Diseñar una interfaz sencilla y fácil de usar, pensada para que cualquier usuario pueda entender rápidamente cómo revisar, comentar y aprobar contenidos, tanto desde ordenador como desde móvil.

8. Organizar el proyecto siguiendo una estructura clara en API REST(Laravel) + SPA (Vue.js), separando backend y frontend para facilitar el mantenimiento, la escalabilidad y la posible incorporación futura de una aplicación móvil.

9. Configurar el entorno de trabajo con Docker para poder desarrollar y probar la aplicación en un entorno controlado y reproducible.

10. Desplegar la aplicación en un VPS de OVH Cloud utilizando Nginx, PHP-FPM y PostgreSQL, y documentar los pasos básicos para su instalación y puesta en marcha.

3.3 Objetivos técnicos y de calidad

- Mantener una estructura de proyecto ordenada y fácil de entender.

- Usar una base de datos relacional PostgreSQL con Eloquent para gestionar la información.

- Añadir validaciones en los formularios para evitar errores y datos incorrectos.

- Implementar un sistema básico de seguridad con inicio de sesión y control de permisos.

- Usar Git para llevar el control de versiones del proyecto.

- Documentar lo más importante del funcionamiento de la aplicación y su estructura.

4. Antecedentes y estado del arte

En el mercado existen varias soluciones para la gestión, revisión y aprobación de contenido y/o activos digitales, algunas más orientadas a producción técnica o grandes empresas, y otras más generalistas. Sin embargo, pocas están pensadas específicamente para el flujo de trabajo entre creadores de contenido y clientes en agencias pequeñas, con **comentarios visuales sencillos, plazos de aprobación y trazabilidad simple**. A continuación se analizan algunas de las principales soluciones comparables.

4.1 Soluciones comparables (estado del arte)

1. **Frame.io**

    - Es una herramienta pensada originalmente para equipos de producción audiovisual y post-producción.

    - Está fuertemente orientada al **trabajo colaborativo sobre vídeo** (review / anotaciones / versiones) y se parece más a un “Figma para vídeo” que a una plataforma

general de gestión de activos.

- Su enfoque es técnico, útil para filmmakers, editores y departamentos de vídeo con flujos complejos.

- Planes desde **$15 por miembro/mes**; ofrece revisión de clips, comentarios por fotograma, comparador de versiones, etc.

- No está específicamente diseñada para workflows de agencia - cliente en contenido social.

- Fuente: frame.io pricing.

2. **Filestage**

- Plataforma de proofing y revisión enfocada a aprobaciones de contenido de marketing.

- Permite comentarios en diferentes tipos de archivos y flujos de aprobación.

- En Capterra muestra buenas valoraciones pero precios desde **€99/mes** en el plan básico.

- Fuente: cápter Capterra Filestage.

3. **Ziflow**

- Software de proofing y aprobación con foco en equipos y agencias de marketing más grandes.

- Alta valoración de usuarios (4.8/5 con cientos de reviews).

- Más orientado a flujos formales de revisión de materiales que a uso independiente.

- Fuente: Capterra Ziflow.

4. **Bynder**

    - Solución completa de Digital Asset Management (DAM) orientada a empresas medianas y grandes.

    - Ofrece repositorio central, flujos de trabajo, integraciones y automatizaciones.

    - Su enfoque es corporativo y no está pensado para herramientas simples de revisión entre creador - cliente.

    - Por su complejidad y precio, suele usarse en entornos enterprise.

    - Valoración en G2 y casos de uso enterprise.

5. **Notion**

    - Herramienta generalista para organizar proyectos, bases de datos, tareas y documentos.

- Muy usada por creadores para organizar contenido, calendarios y entregables, pero **no es DAM ni tiene funciones dedicadas de revisión/aprobación visual**.

- Más barato y flexible, pero obliga a "hacer malabares" con varias herramientas.

4.2 Tabla comparativa

Resumen: Las soluciones existentes cubren partes de lo que Contentio pretende resolver, pero o bien son **técnicas y caras**, o bien **genéricas y no especializadas**. No existe una herramienta ligera, asequible y específica que combine revisión visual, trazabilidad y flujos de aprobación simples para creadores y sus clientes.

Plataforma	Enfoque principal	Características clave	Licencia / modelo	Pros	Contras
Frame.io	Revisión colaborativa de vídeo	Comentarios por fotograma, comparador de versiones	SaaS por usuario	Muy potente para vídeo técnico	No es DAM general; orientado a producción audiovisual
Filestage	Proofing y aprobación de contenido	Flujos de aprobación, comentarios	SaaS por plan mensual	Buena gestión de revisiones	Coste relativamente alto para autónomos

Ziflow	Proofing para marketing/agencias	Gestión de pruebas, aprobaciones	SaaS	Muy valorado por agencias	Puede ser complejo/sobre-dimensionado
Bynder	DAM empresarial	Gestión centralizada de activos, integraciones	SaaS/Enterprise	Solución completa DAM	Coste y complejidad altos
Notion	Organización generalista	Bases de datos, tareas	SaaS (freemium)	Barato y flexible	No está pensado para revisión visual ni trazabilidad real

4.3 Análisis DAFO

A continuación se muestra un análisis DAFO que resume las **fortalezas, debilidades, oportunidades y amenazas** de Contentio como proyecto:

Esquema DAFO (cuadrante)

Debilidades	**Amenazas**
● Producto nuevo sin reputación ● MVP limitado en funcionalidades	● Competidores con más recursos ● Herramientas generalistas siguen

● Coste de almacenamiento ● Necesidad de validación real	siendo populares ● Cambios en hábitos de trabajo ● Competidores podrían bajar precios / ampliar mercado
Fortalezas	**Oportunidades**
● Enfoque específico en flujo creador - cliente. ● Comentarios visuales sencillos tipo Figma ● Plazos de aprobación con auto-ok ● Interfaz más accesible ● Producto más asequible	● Mercado creciente de creadores para negocios locales. ● Escalable a nichos similares. ● Potencial para monetización

4.4 ¿Cómo ganar dinero con esto?

Una estrategia de monetización razonable para Contentio sería:

Modelo de suscripción mensual escalable:

- **Starter** – para creadores con pocos clientes
- **Pro** – más clientes y mayor almacenamiento
- **Team/Agency** – multi-usuarios, branding y control más completo

Add-ons opcionales:

- Almacenamiento extra de archivos
- Retención de histórico más largo
- Marca blanca para agencias

Referencias y datos contrastables

- **Frame.io**: planes desde $15/mes por miembro.
  Fuente: https://frame.io/pricing
- **Filestage**: valoraciones y precios básicos desde €99/mes.
  Fuente: https://www.capterra.com/p/162035/Filestage/
- **Ziflow**: 4.8/5 con más de 400 reseñas en Capterra.
  Fuente: https://www.capterra.com/p/178111/Ziflow/
- **Notion**: más de 100 millones de usuarios según su blog oficial.
  Fuente: https://www.notion.com/blog/100-million-of-you
- **Bynder**: plataforma DAM con presencia en G2 y valoraciones en entorno enterprise.

5. Análisis y especificación de requisitos

En este apartado se definen los requisitos del sistema Contentio, separando los **requisitos funcionales** (qué debe hacer la aplicación) y los **no funcionales** (cómo debe comportarse). Además, se organizan en forma de **épicas, historias de usuario y tareas**, se representan **casos de uso y diagrama de actividad**, se añaden **criterios de aceptación** y se incluye una **estimación de esfuerzo** agrupada por módulos.

5.1 Requisitos funcionales

Para organizar el desarrollo, los requisitos se estructuran en **épicas**, que se dividen en **historias de usuario**, y estas a su vez en **tareas**.

Épica 1: Gestión de usuarios y roles

Historia 1.1: Como usuario, quiero poder registrarme e iniciar sesión para acceder a la plataforma.

Tareas:

- Crear formulario de registro.
- Crear formulario de inicio de sesión.
- Validar credenciales.
- Gestionar cierre de sesión.

Historia 1.2: Como administrador, quiero poder asignar roles (agencia, creador, cliente) para controlar qué puede hacer cada usuario.

Tareas:

- Definir roles en el sistema.
- Restringir accesos según rol.
- Crear pantallas básicas de gestión de usuarios.

Épica 2: Gestión de clínicas y contenidos

Historia 2.1: Como creador, quiero poder crear y gestionar clínicas para organizar mis clientes.

Tareas:

- Crear formulario de alta de clínica.

- Listar clínicas.

- Editar y eliminar clínicas.

Historia 2.2: Como creador, quiero poder crear contenidos asociados a una clínica.
Tareas:

- Crear formulario de nuevo contenido.

- Asociar contenido a clínica.

- Listar contenidos por clínica.

Épica 3: Subida y gestión de archivos

Historia 3.1: Como creador, quiero poder subir imágenes y vídeos a un contenido para que el cliente los revise.
Tareas:

- Implementar subida de archivos.

- Guardar rutas y metadatos en la base de datos.

- Mostrar previsualización del archivo.

Épica 4: Revisión, comentarios y aprobaciones

Historia 4.1: Como cliente, quiero poder dejar comentarios sobre un contenido para pedir cambios.
Tareas:

- Crear sistema de comentarios por contenido.

- Mostrar comentarios en la vista del contenido.

- Guardar fecha y autor del comentario.

Historia 4.2: Como cliente, quiero poder marcar puntos concretos de mejora sobre una imagen o material, de forma visual.

Tareas:

- Permitir añadir marcas sobre la imagen.

- Asociar comentarios a esas marcas.

- Mostrar esas marcas al creador.

Historia 4.3: Como creador, quiero poder cambiar el estado de un contenido (borrador, en corrección, listo para revisión).

Tareas:

- Definir estados posibles.

- Crear botones de cambio de estado.

- Guardar historial de cambios.

Historia 4.4: Como creador, quiero definir un plazo máximo de aprobación para que, si el cliente no responde a tiempo, el contenido se marque como aprobado automáticamente.

Tareas:

- Añadir campo de fecha límite.

- Comprobar vencimientos.

- Implementar aprobación automática.

5.2 Requisitos no funcionales

- **Rendimiento:** La aplicación debe responder de forma fluida en operaciones comunes como listar contenidos o añadir comentarios.

- **Seguridad:**

    - El sistema debe requerir autenticación para acceder.

    - Cada usuario solo podrá acceder a lo que le permita su rol.

- **Usabilidad:**

    - La interfaz debe ser sencilla y fácil de entender.

    - Debe poder usarse tanto en ordenador como en móvil.

- **Accesibilidad:**

    - Se procurará usar un diseño claro, con textos legibles y buena organización visual.

- **Internacionalización (i18n):**

    - La aplicación quedará preparada para poder mostrar textos en más de un idioma en el futuro.

- **Testabilidad:**

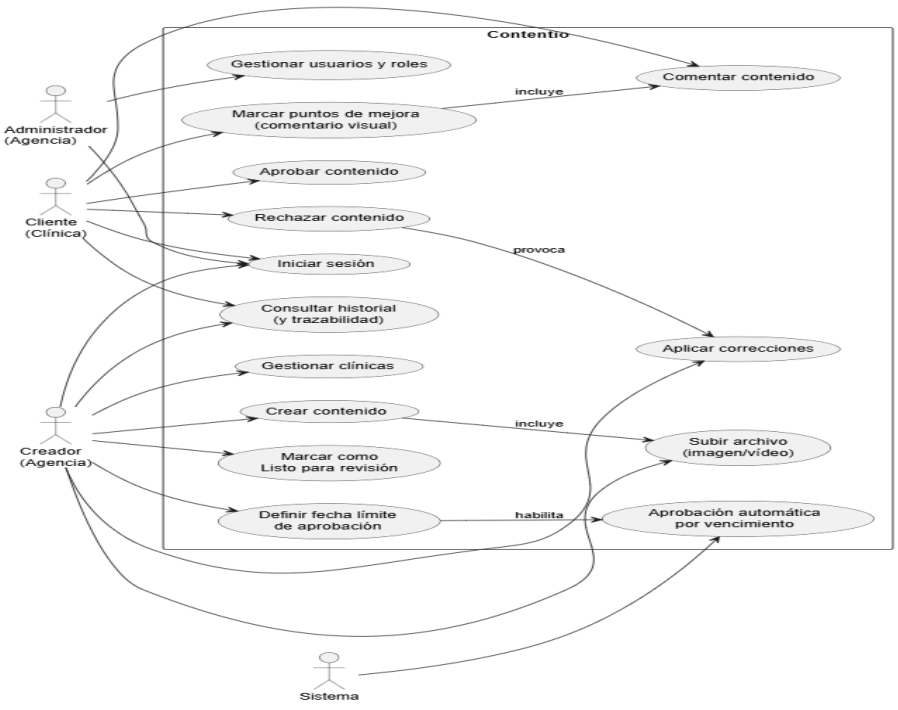
    - El código deberá estar organizado de forma que permita añadir pruebas más adelante.

- **Despliegue:**

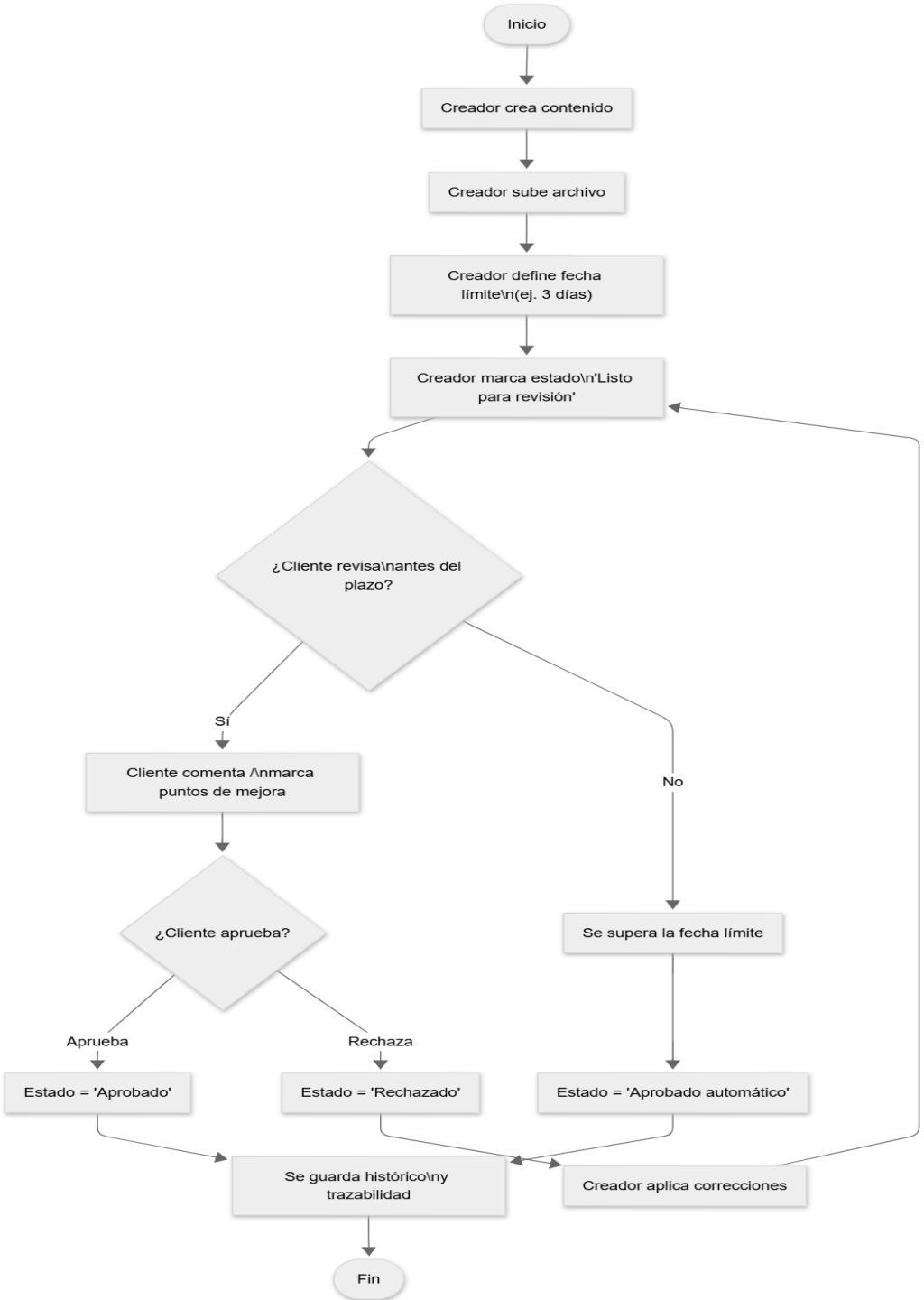
    - La aplicación deberá poder desplegarse en un VPS de OVH Cloud usando Docker.

5.3 Casos de uso

Algunos de los principales casos de uso del sistema son:



5.4 Diagrama de actividad



5.5 Criterios de aceptación

Historia 4.1 – Comentarios en contenido

- Dado que el usuario está autenticado
- Cuando entra en un contenido
- Entonces puede escribir un comentario y guardarlo
- Y el comentario aparece en la lista con su nombre y fecha

Historia 4.4 – Aprobación automática por vencimiento

- Dado que un contenido tiene una fecha límite de aprobación
- Cuando se supera esa fecha sin respuesta del cliente
- Entonces el sistema cambia el estado del contenido a “aprobado” automáticamente

5.6 Estimación y agrupación por módulos

Los requisitos se agrupan en los siguientes módulos:

- **Módulo 1:** Usuarios y roles
- **Módulo 2:** Clínicas y contenidos
- **Módulo 3:** Archivos multimedia

- **Módulo 4:** Revisión, comentarios y aprobaciones

- **Módulo 5:** Infraestructura y despliegue

Ejemplo de estimación (orientativa):

Módulo	**Dificultad**	**Tiempo estimado**
Usuarios, roles y autenticación	Media	8–10 horas
Clínicas y contenidos	Media	10–12 horas
Archivos multimedia	Media	8–10 horas
Revisión, comentarios y anotaciones	Alta	12–14 horas
Frontend Vue.js (Vistas + componentes)	Alta	12-14 horas
Infraestructura y despliegue	Media	8–10 horas
Total aproximado		**58-70 horas**

6. Propuesta de solución y diseño

6.1 Arquitectura

6.1.1 Arquitectura por capas

Contentio se organiza como una aplicación con arquitectura API REST + SPA: el backend (Laravel) expone una API JSON y el frontend (Vue.js) es una Single Page Application independiente que consume esa API.

- Backend (API): Laravel 11 — controladores API, servicios (workflow, almacenamiento, notificaciones), Eloquent ORM, Laravel Sanctum (autenticación).
- Frontend (SPA): Vue.js 3 — componentes reactivos, Vue Router (navegación), Pinia (estado), Axios (comunicación HTTP),Tailwind CSS (estilos).
- Datos: PostgreSQL + almacenamiento local de archivos (preparado para S3 a futuro mediante Laravel Storage).

Diagrama de capas:



6.1.2 Flujo de navegación

Creador (agencia):

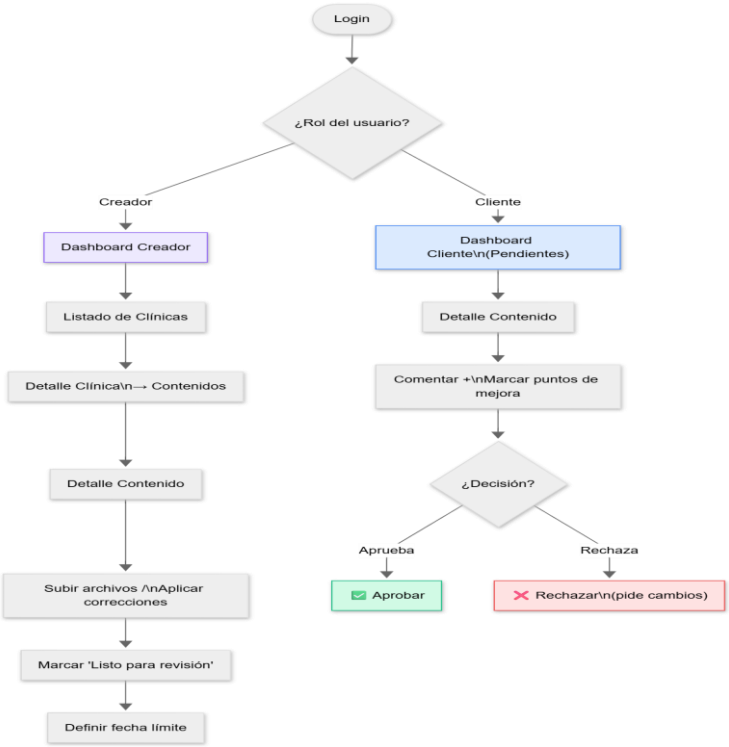
- **Login → Dashboard → Clínicas → Contenidos → Detalle contenido**

- **Acciones: crear contenido, subir archivos, aplicar correcciones, marcar "Listo para revisión", definir plazo**

Cliente (clínica):

- **Login → Dashboard (pendientes) → Detalle contenido**

- **Acciones: comentar, marcar puntos de mejora, aprobar o rechazar**

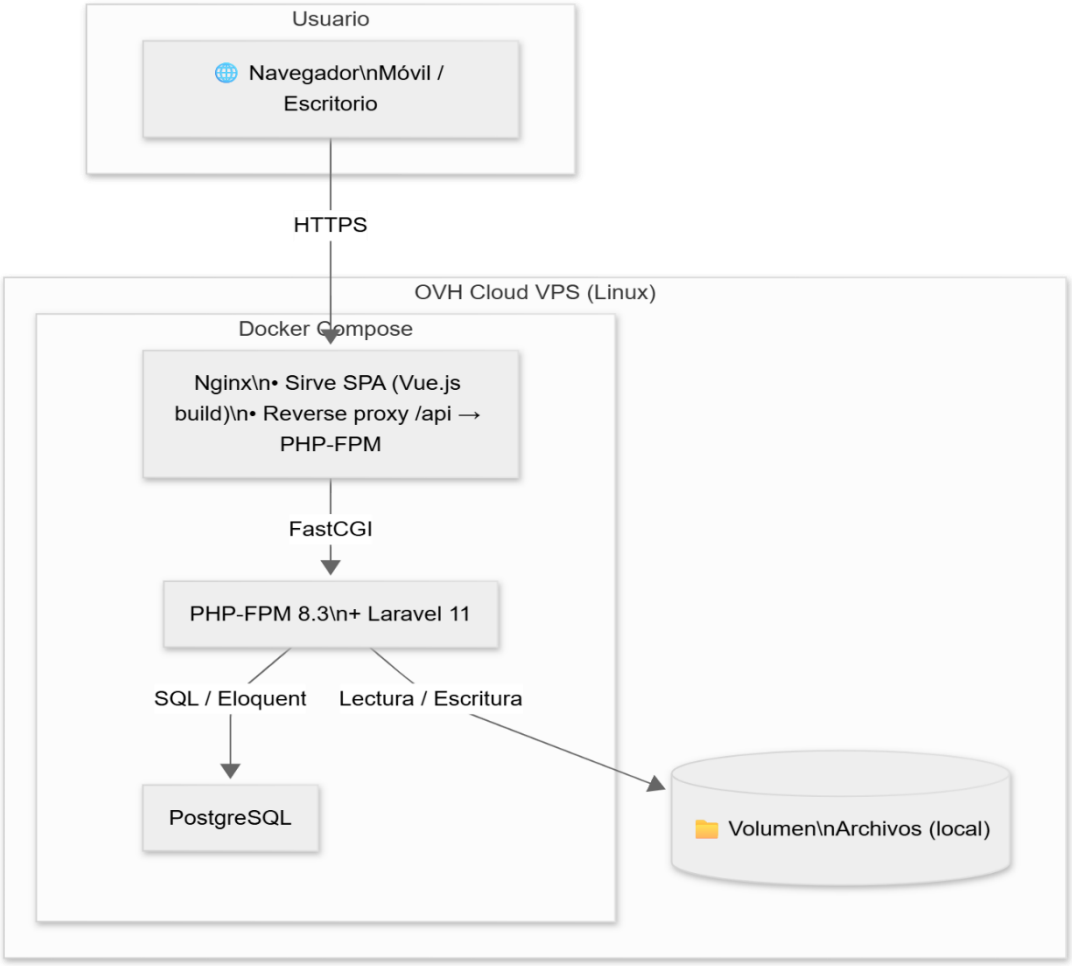


6.1.3 Despliegue

La aplicación se desplegará en un **VPS de OVH Cloud** con Docker Compose:

- Nginx como servidor web
- PHP-FPM ejecutando Laravel

  PostgreSQL como base de datos
- Almacenamiento de archivos en disco (MVP), dejando preparada la migración a Object Storage (S3 compatible)



6.2 Tecnologías multiplataforma y justificación

Se opta por una arquitectura API REST + SPA por las siguientes razones:

1. Multiplataforma real: La API REST permite que en el futuro se conecte una app móvil sin modificar el backend. En el MVP, la SPA en Vue.js cubre escritorio y móvil mediante diseño responsive con Tailwind CSS.

2. Separación de responsabilidades: El backend se ocupa de la lógica de negocio, validaciones y persistencia. El frontend se encarga de la experiencia de usuario.

3. Interactividad: Las funcionalidades clave de Contentio (anotaciones visuales con pines sobre imágenes, cambios de estado, panel de comentarios) requieren un frontend reactivo que Vue.js proporciona de forma natural.

4. Productividad: Laravel ofrece un ecosistema integrado (Sanctum, Storage, Eloquent, Scheduler) que permite desarrollar un MVP funcional en el tiempo disponible. Vue.js 3 con Composition API proporciona una arquitectura de componentes clara y reutilizable.

Se utiliza:

- Laravel 11 (PHP 8.3) + Eloquent ORM para el backend API

- Vue.js 3 + Vue Router + Pinia + Axios para el frontend SPA

- Tailwind CSS para el diseño responsive

- PostgreSQL para la persistencia

- Docker + Docker Compose para entorno reproducible

- OVH Cloud VPS como infraestructura

6.3 Modelado: dominio y base de datos

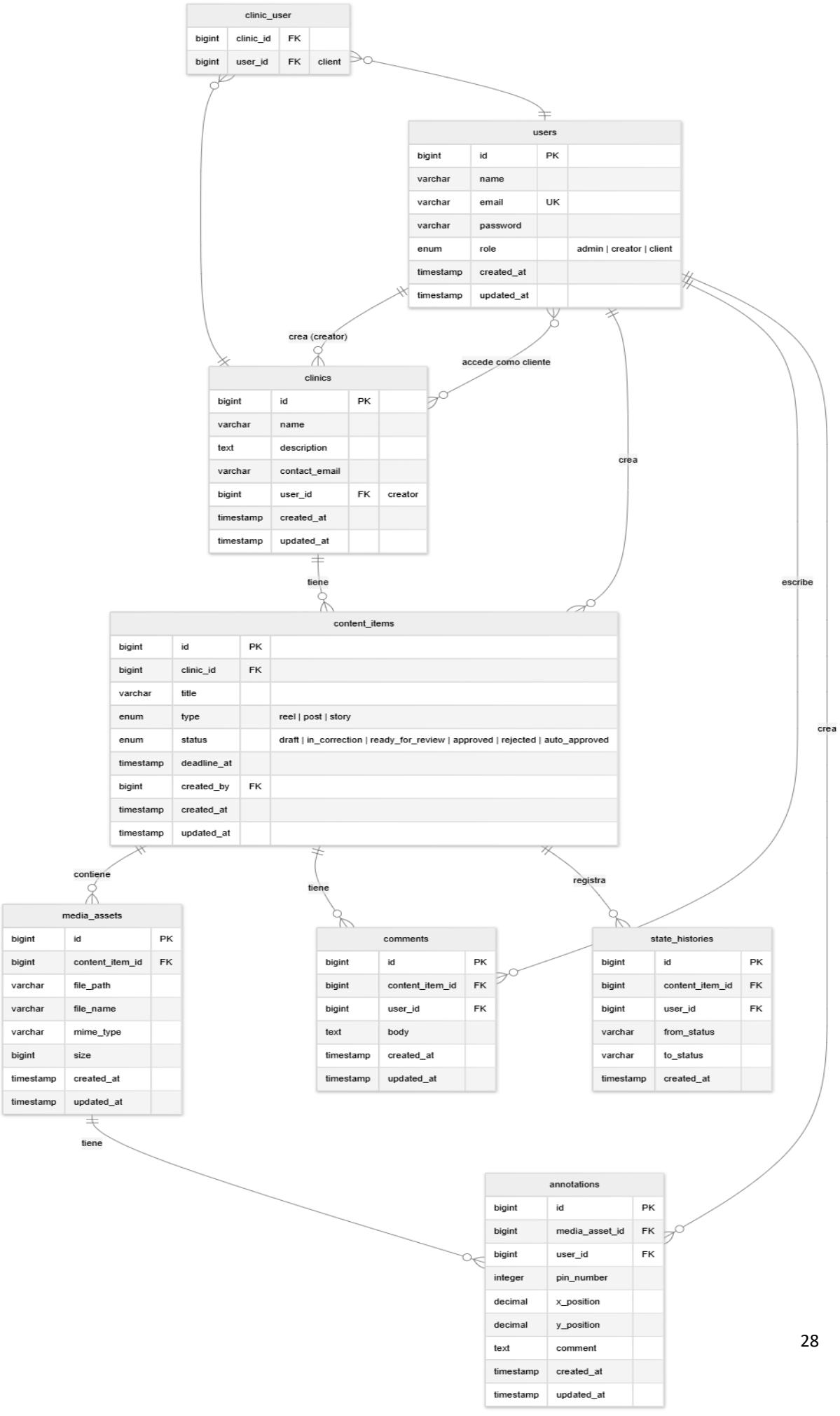
6.3.1 Modelo de dominio

En el MVP se manejarán principalmente las siguientes entidades:

- **User** (roles: admin, creador, cliente)
- **Clinic**
- **ContentItem** (tipo: Reel, Post, Story)
- **MediaAsset** (archivo: imagen/vídeo, ruta, metadatos)
- **Comment**
- **Annotation** (marca visual sobre la imagen + comentario)
- **StateHistory** (historial de cambios de estado)

Regla importante del tipo de contenido:

- El contenido se clasifica como **Reel, Post o Story**, porque es el formato real con el que trabajan las clínicas.


clinic_user
bigint clinic_id FK
bigint user_id FK client
users
bigint id PK
varchar name
varchar email UK
varchar password
enum role admin | creator | client
timestamp created_at
timestamp updated_at
crea (creator)
accede como cliente
clinics
bigint id PK
varchar name
text description
varchar contact_email
bigint user_id FK creator
timestamp created_at
timestamp updated_at
crea
tiene
escribe
content_items
bigint id PK
bigint clinic_id FK
varchar title
enum type reel | post | story
enum status draft | in_correction | ready_for_review | approved | rejected | auto_approved
timestamp deadline_at
bigint created_by FK
timestamp created_at
timestamp updated_at
crea
contiene
tiene
registra
media_assets
bigint id PK
bigint content_item_id FK
varchar file_path
varchar file_name
varchar mime_type
bigint size
timestamp created_at
timestamp updated_at
comments
bigint id PK
bigint content_item_id FK
bigint user_id FK
text body
timestamp created_at
timestamp updated_at
state_histories
bigint id PK
bigint content_item_id FK
bigint user_id FK
varchar from_status
varchar to_status
timestamp created_at
tiene
annotations
bigint id PK
bigint media_asset_id FK
bigint user_id FK
integer pin_number
decimal x_position
decimal y_position
text comment
timestamp created_at
timestamp updated_at

6.3.2 Estructura de datos (DB)

Tablas previstas (MVP):

- users
- clinics
- clinic user
- content_items (incluye type y deadline)
- media_assets
- comments
- annotations
- state_history

6.4 UX/UI: wireframes (Figma) clave + guía de estilos

La interfaz se diseñará para que sea clara y rápida de usar tanto en móvil como en escritorio. Se prioriza que el cliente pueda revisar y aprobar sin complicaciones, y que el creador tenga una vista ordenada del trabajo pendiente y del histórico. Para acelerar el desarrollo del MVP y mantener consistencia visual, se utilizará **Tailwind CSS**.

Wireframes clave (Figma):

- Login
- Dashboard creador (clínicas, contenidos y estados)

- Dashboard cliente (pendientes de revisión)

- Detalle de contenido (archivo + comentarios + historial)

- Vista de revisión (aprobar/rechazar + fecha límite)

- Vista de correcciones (creador)



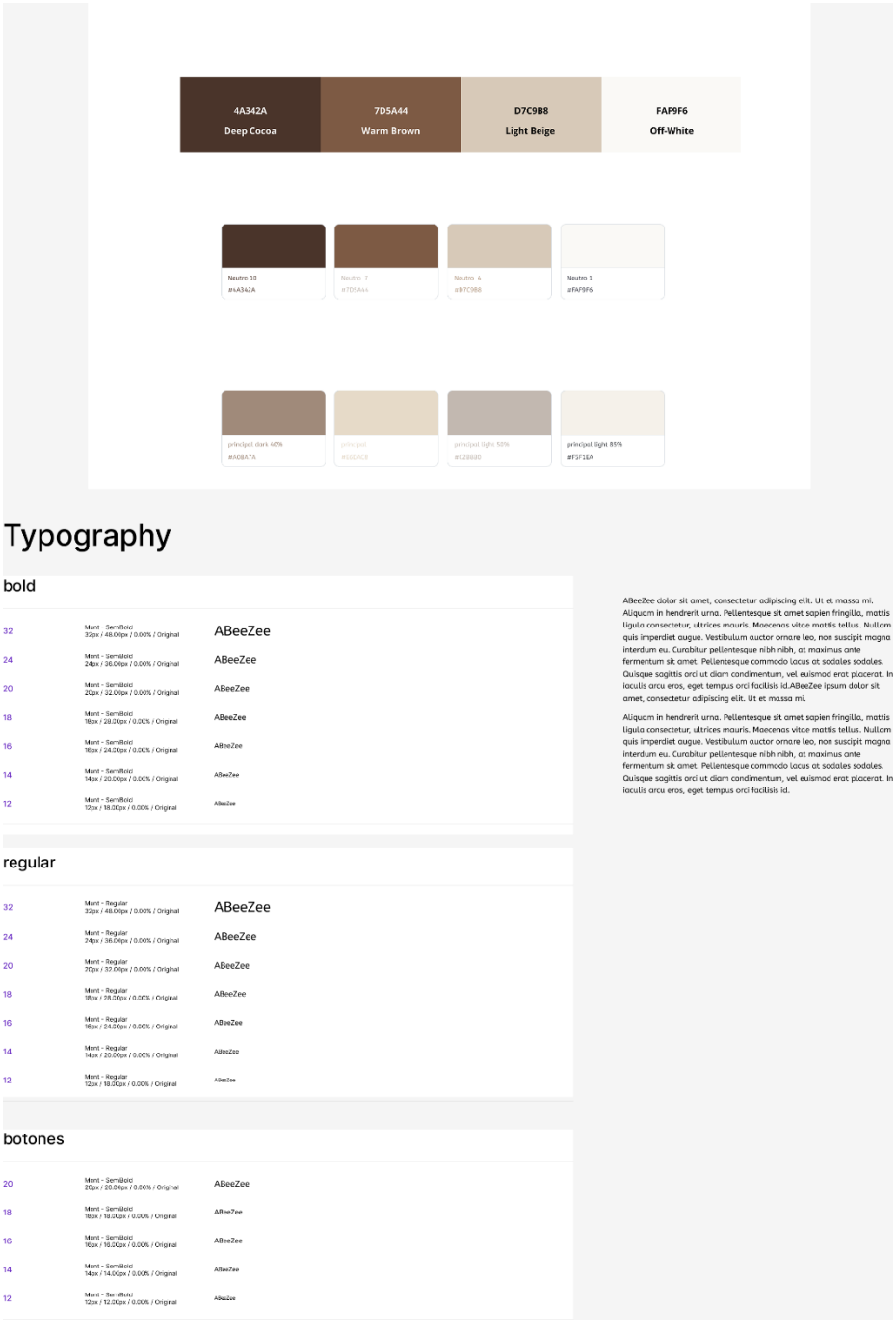
Guía de estilos (simple):

- Diseño limpio, con jerarquía clara de títulos, botones y estados.

- Estados visibles mediante “badges” (borrador, listo para revisión, aprobado, rechazado, aprobado automático).

- Componentes base: cards, botones, timeline de historial, panel de comentarios.

- Espaciado consistente y formularios fáciles de entender.



Anotaciones visuales en imágenes (pines numerados)

Para mejorar la comunicación durante la revisión, Contentio permitirá añadir anotaciones visuales sobre imágenes mediante un sistema de **pines numerados**, similar a lo que se ve en herramientas tipo Figma, pero de forma más sencilla. El cliente podrá hacer clic en una zona concreta de la imagen para crear un pin (por ejemplo, 1, 2, 3…), y cada pin tendrá asociado un comentario de texto.

Así, en vez de decir "cambia lo de arriba a la derecha", el cliente puede marcar exactamente el punto y dejar el comentario correspondiente. Esto reduce malentendidos y acelera el proceso de corrección.

Reglas de esta funcionalidad (MVP):

- Las anotaciones visuales estarán disponibles solo para **contenidos de tipo imagen**.
- Cada anotación se verá como un **pin numerado** sobre la imagen.
- Cada pin tendrá un comentario de texto asociado.
- La posición del pin se guardará mediante coordenadas sobre la imagen.
- Todo quedará registrado con usuario y fecha como parte del histórico.

Este enfoque permite señalar cambios de forma visual y directa, mejorando la comunicación entre cliente y creador sin convertir la plataforma en una herramienta compleja de edición.

6.5 Lógica de negocio real: reglas, validaciones y procesos

La plataforma se basa en un flujo simple y realista de trabajo entre creador y cliente:

Roles

- El **creador** prepara y sube el contenido, aplica correcciones y lo envía a revisión.

- El **cliente** revisa y decide: aprueba o rechaza.

- El **sistema** puede aprobar automáticamente si vence el plazo sin respuesta.

Estados del contenido (MVP)

- Borrador

- En corrección

- Listo para revisión

- Aprobado

- Rechazado

- Aprobado automático (por vencimiento)

Plazo de aprobación

- El creador podrá definir una fecha límite para que el cliente apruebe o rechace.

- El plazo será configurable, con un valor por defecto de **3 días**.

- Si el cliente no realiza ninguna acción antes de que venza el plazo, el sistema marcará el contenido como **Aprobado automático** y dejará registro en el historial.

Trazabilidad

- Cada comentario, pin, cambio de estado y modificación relevante se registra con usuario y fecha.

- Se mantiene un historial consultable para que quede evidencia del trabajo realizado (especialmente útil en contenidos tipo stories).

6.6 Ajuste de tamaño

El proyecto se centra en un MVP que cubre el ciclo completo: creación > revisión > comentarios > aprobación/rechazo > histórico. Esto evita que sea un proyecto trivial. Al mismo tiempo, se dejan fuera integraciones avanzadas (publicación automática en redes, analítica, app móvil nativa), lo que hace que el alcance sea realista y viable dentro del marco temporal del TFG.

SECCIÓN 7 — Plan de trabajo (previsto)

7.1 Roadmap por sprints

El proyecto se desarrolla en 6 sprints de 1-2 semanas entre la aprobación del anteproyecto y la entrega final, organizados en torno a los tres hitos de evaluación del centro.

Sprint: Sprint 0 — Setup

Período: 22/02 – 28/02

Contenido: Entorno Docker, estructura base Laravel + Vue.js, repositorio Git, esquema PostgreSQL

Hito: —

Sprint: Sprint 1 — Auth y usuarios

Período: 01/03 – 08/03

Contenido: Autenticación con Laravel Sanctum, roles agencia/clínica, dashboards diferenciados

Hito: —

Sprint: Sprint 2 — Gestión de contenidos

Período: 09/03 – 15/03

Contenido: CRUD de contenidos (reels, posts, stories), subida de archivos con Laravel Storage, máquina de estados básica

Hito: 50% MVP (15/03)

Sprint: Sprint 3 — Revisión y comentarios

Período: 16/03 – 27/03

Contenido: Sistema de anotaciones visuales (pines), comentarios por contenido, historial de estados

Hito: —

Sprint: Sprint 4 — Flujo de aprobación

Período: 28/03 – 05/04

Contenido: Flujo completo aprobación/rechazo, aprobación automática por plazo configurable, trazabilidad

Hito: 85% Beta (05/04)

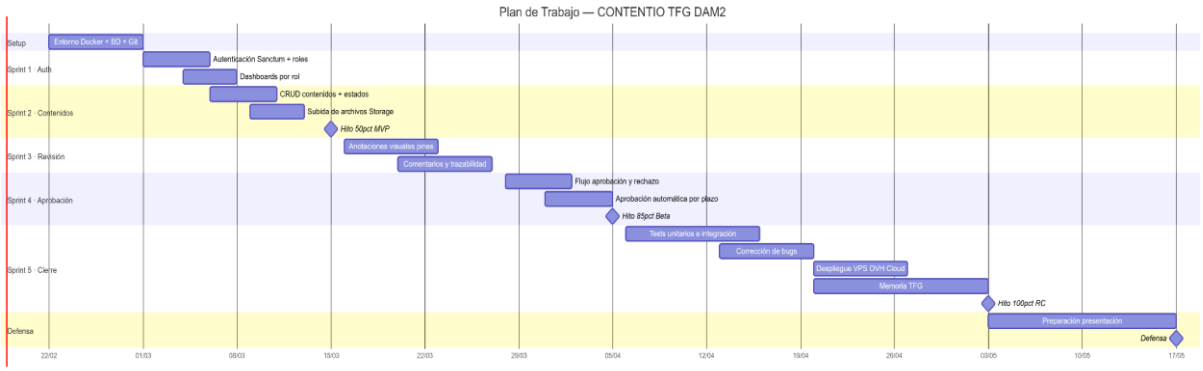
Sprint: Sprint 5 — Pruebas y cierre

Período: 06/04 – 03/05

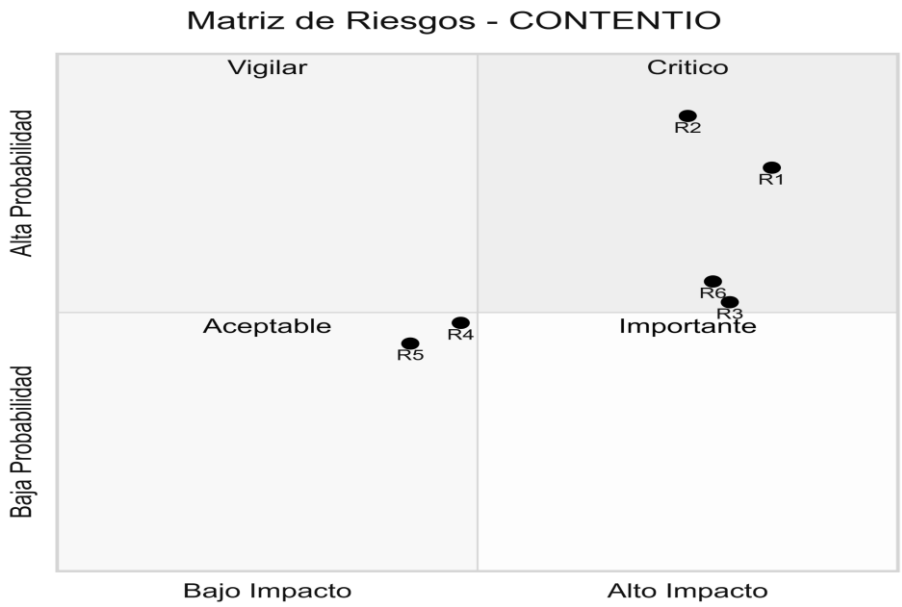
Contenido: Tests unitarios e integración, corrección de bugs, despliegue en VPS OVH Cloud, memoria final

Hito: 100% RC (03/05)

7.2 Diagrama de Gantt



7.3 Matriz de riesgos



#: R1

Riesgo: Complejidad del sistema de anotaciones visuales (pines)

Prob.: Alta

Impacto: Alto

Mitigación: Usar librería canvas existente (Fabric.js); simplificar a marcadores de zona si es necesario

#: R2

Riesgo: Plazos muy ajustados entre entregas

Prob.: Alta

Impacto: Alto

Mitigación: Priorizar funcionalidades core del MVP; reducir scope en características secundarias

#: R3

Riesgo: Gestión de archivos de vídeo pesados

Prob.: Media

Impacto: Alto

Mitigación: Limitar tamaño máximo de archivo en el MVP; diferir procesamiento de vídeo a versión posterior

#: R4

Riesgo: Configuración CORS en SPA con Sanctum

Prob.: Media

Impacto: Medio

Mitigación: Seguir documentación oficial Laravel; usar boilerplate probado desde el Sprint 0

#: R5

Riesgo: Despliegue Docker en VPS OVH

Prob.: Media

Impacto: Medio

Mitigación: Preparar docker-compose.prod.yml desde el inicio; validar en entorno de staging antes de la entrega

#: R6

Riesgo: Disponibilidad de tiempo por carga académica/laboral

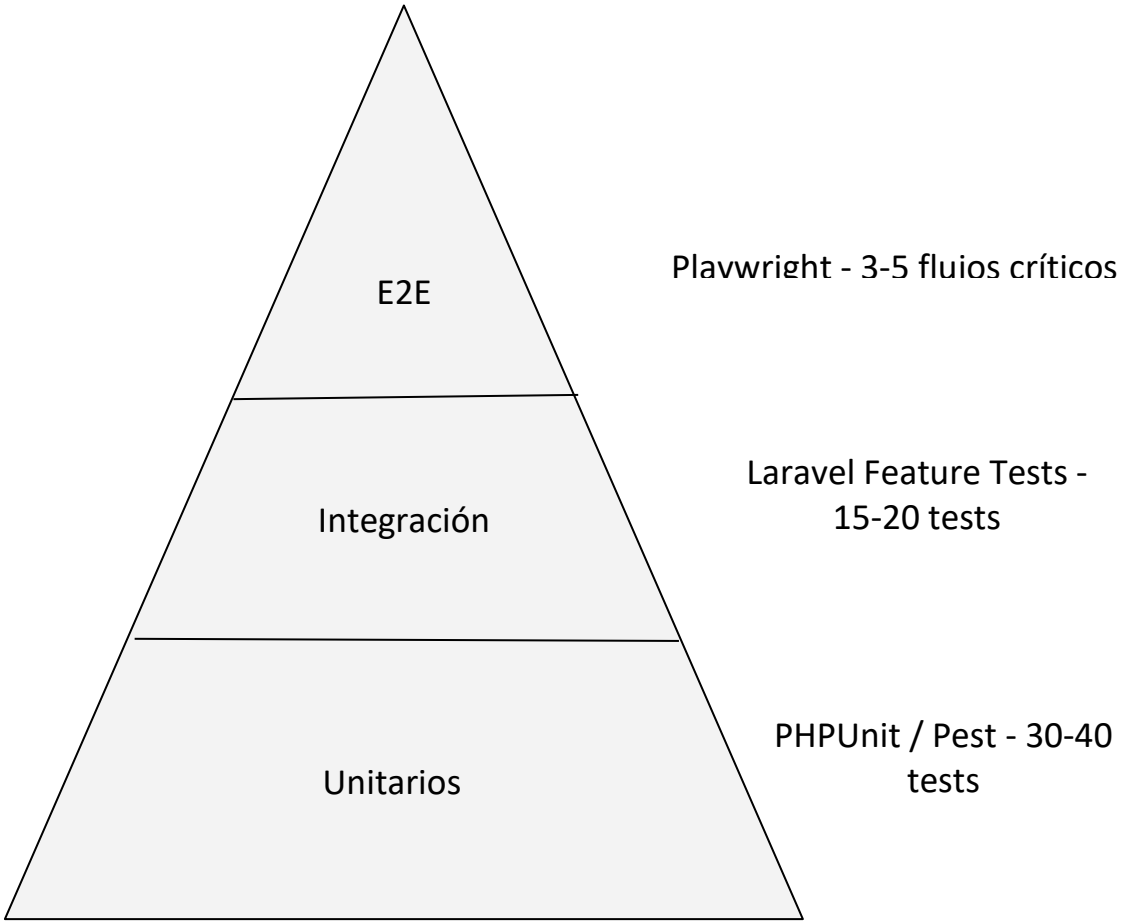
Prob.: Media

Impacto: Alto

Mitigación: Reservar buffer en Sprint 5; el alcance del MVP está deliberadamente acotado para ser abarcable

SECCIÓN 8 — Plan de pruebas y calidad

8.1 Pirámide de tests



Nivel: Unitarios

Herramienta: PHPUnit / Pest

Qué se prueba: Servicios de negocio, modelos, validaciones

Cobertura objetivo: ≥ 70% backend

Nivel: Integración

Herramienta: Laravel Feature Tests

Qué se prueba: Endpoints API REST (auth, CRUD, estados)

Cobertura objetivo: Todos los endpoints del MVP

Nivel: E2E / UI

Herramienta: Playwright + Vue Test Utils

Qué se prueba: Flujos críticos completos de usuario

Cobertura objetivo: 3-5 flujos críticos

8.2 Testing manual — Checklist

ID: TM-01

Flujo: Login agencia

Datos de prueba: agencia@test.com / Test1234!

Resultado esperado: Redirige a Dashboard Creador

ID: TM-02

Flujo: Login clínica

Datos de prueba: clinica@test.com / Test1234!

Resultado esperado: Redirige a Dashboard Cliente

ID: TM-03

Flujo: Subir contenido

Datos de prueba: JPG < 5 MB, título "Post prueba"

Resultado esperado: Aparece en lista con estado Borrador

ID: TM-04

Flujo: Cambio a "Listo para revisión"

Datos de prueba: Contenido en Borrador

Resultado esperado: Estado cambia, cliente lo ve disponible

ID: TM-05

Flujo: Añadir pin de anotación

Datos de prueba: Clic sobre imagen en vista revisión

Resultado esperado: Pin numerado visible, formulario activo

ID: TM-06

Flujo: Aprobar contenido

Datos de prueba: Contenido en revisión

Resultado esperado: Estado → Aprobado, queda en historial

ID: TM-07

Flujo: Rechazar con comentario

Datos de prueba: Contenido en revisión

Resultado esperado: Estado → Rechazado, comentario visible

ID: TM-08

Flujo: Aprobación automática

Datos de prueba: Contenido sin respuesta 3 días

Resultado esperado: Estado → Aprobado automáticamente

8.3 CI local

Scripts de ejecución local para verificar la suite antes de cada commit:

```
# Backend — cobertura mínima 70%
php artisan test --coverage --min=70

# Frontend — tests de componentes
npm run test:unit

# E2E — flujos críticos
npx playwright test
```

8.4 Definition of Done

Una historia de usuario se considera completada cuando:

- El código compila sin errores y sin warnings críticos
- Los tests unitarios relacionados pasan con cobertura ≥ 70%
- Los tests de integración del endpoint afectado pasan
- La funcionalidad ha sido verificada con el checklist manual
- No hay regresiones en el resto de la suite
- El commit está en la rama correspondiente con mensaje descriptivo
- El caso de uso queda cubierto en la matriz de trazabilidad

8.5 Matriz de trazabilidad

Objetivo	**Historia de usuario**	**Casos de prueba**
OBJ-01 Roles y autenticación	HU-01 Registro e identificación por rol	UT-01, IT-01, TM-01, TM-02
OBJ-02 Gestión de contenidos	HU-02 Crear y subir contenido	UT-05, IT-03, TM-03, TM-04

OBJ-03 Revisión con anotaciones	HU-03 Revisar contenido con pines	UT-08, IT-05, TM-05
OBJ-04 Flujo de aprobación	HU-04 Aprobar o rechazar contenido	UT-10, IT-07, TM-06, TM-07
OBJ-05 Trazabilidad completa	HU-05 Historial de estados	UT-12, IT-08
OBJ-06 Aprobación automática	HU-06 Configurar plazo de aprobación	UT-14, IT-09, TM-08

9. Estrategia de despliegue e instalación

9.1 Requisitos mínimos

CONTENTIO es una aplicación web SaaS contenerizada con Docker. No requiere instalación en el dispositivo del usuario

final: basta con un navegador moderno. El entorno de producción necesita:

Servidor (VPS OVH Cloud o equivalente):

Recurso	Mínimo	Recomendado

CPU	2 vCPU	4 vCPU
RAM	2 GB	4 GB
Almacenamiento	20 GB SSD	40 GB SSD
SO	Ubuntu 22.04 LTS	Ubuntu 22.04 LTS
Red	Puerto 80 y 443 abiertos	+ dominio propio

Dependencias del servidor:

- Docker Engine 24+
- Docker Compose v2 (plugin integrado)
- Git 2.x
- Certbot / Let's Encrypt (HTTPS)

Cliente (usuario final):

- Navegador moderno: Chrome 100+, Firefox 110+, Safari 16+, Edge 100+
- Sin instalación adicional

9.2 Procedimiento de despliegue

El despliegue se realiza mediante Docker Compose, que orquesta tres contenedores: nginx (proxy y servicio de archivos estáticos), php-fpm (backend Laravel) y postgres (base de datos).

Paso 1 — Clonar el repositorio y configurar el entorno

- git clone https://github.com/<usuario>/contentio.git
- cd contentio
- cp .env.example .env

Las variables de entorno críticas a configurar en .env:

Variable	Descripción
APP_KEY	Clave de cifrado Laravel (generada con php artisan key:generate)
APP_URL	URL pública del servidor (p.ej. https://contentio.app)
DB_DATABASE, DB_USERNAME, DB_PASSWORD	Credenciales PostgrSQL
SANCTUM_STATEFUL_DOMAINS	Dominio del frontend (para autenticación SPA)

FILESYSTEM_DISK	local (desarrollo) o s3 (producción con almacenamiento externo)

Paso 2 — Build del frontend

E l contenedor node solo se usa en fase de build; genera el bundle estático en/frontend/dist/, que Nginx sirve directamente:

```
docker compose run --rm node npm run build
```

Paso 3 — Levantar los servicios

```
docker compose up -d --build
```

Esto inicia los tres servicios en segundo plano: nginx, php-fpm y postgres.

Paso 4 — Inicialización de la base de datos

```
# Ejecutar migraciones

docker compose exec php-fpm php artisan migrate --force

# Semillado inicial (roles, permisos y usuario administrador)

docker compose exec php-fpm php artisan db:seed --class=ProductionSeeder
```

El ProductionSeeder crea únicamente los registros imprescindibles para que la aplicación arranque: los tres roles del sistema (admin, creator, client) y el primer usuario administrador con credenciales configurables via .env.

Paso 5 — Configuración final

Enlace simbólico para archivos subidos (storage public)

docker compose exec php-fpm php artisan storage:link

Caché de configuración y rutas (mejora el rendimiento)

docker compose exec php-fpm php artisan config:cache

docker compose exec php-fpm php artisan route:cache

9.3 Publicación y acceso

CONTENTIO es una aplicación web; no se publica en tiendas de aplicaciones. El acceso se realiza directamente a través del navegador mediante la URL pública del servidor.

HTTPS con Let's Encrypt:

certbot --nginx -d contentio.app -d www.contentio.app

Guía de primer acceso para el administrador:

1. Acceder a https://<dominio> desde el navegador.
2. Iniciar sesión con las credenciales del administrador configuradas durante el semillado.
3. Crear la primera clínica y asignar los usuarios creator y client.
4. El sistema está listo para operar.

Los usuarios finales (agencia creativa y clínica) reciben únicamente la URL y sus credenciales; no necesitan instalar nada.

10. Evolución y trabajo futuro

La versión 1.0 de CONTENTIO cubre el flujo core de revisión y aprobación de contenidos. Las siguientes versiones amplían la plataforma de forma incremental, priorizando funcionalidades validadas con usuarios reales durante el período de prueba.

10.1 Backlog de mejoras (v2 y v3)

Versión 2.0 — Productividad y colaboración

Objetivo: reducir la fricción del flujo de trabajo entre agencia y clínica.

ID: F-01

Mejora: Notificaciones en tiempo real (WebSockets con Laravel Reverb)

Alcance: Backend + Frontend

Coste estimado: 16–20 h

ID: F-02

Mejora: Comentarios con hilos (respuestas anidadas)

Alcance: Backend + Frontend

Coste estimado: 10–14 h

ID: F-03

Mejora: Exportación de informe de revisión (PDF con historial de cambios)

Alcance: Backend (PDF)

Coste estimado: 12–16 h

ID: F-04

Mejora: Vista calendario de publicaciones

Alcance: Frontend (Vue)

Coste estimado: 14–18 h

ID: F-05

Mejora: Autoaprobación configurable por clínica (plazo en días)

Alcance: Backend (lógica negocio)

Coste estimado: 8–10 h

Total v2.0 estimado: 60–78 horas de desarrollo.

Versión 3.0 — Escalado y automatización

Objetivo: convertir CONTENTIO en una plataforma multitenant escalable para agencias con múltiples clientes.

ID: F-06

Mejora: Multitenancy completo (aislamiento por tenant en PostgreSQL con schemas)

Alcance: Backend (arquitectura)

Coste estimado: 40–60 h

ID: F-07

Mejora: Integración con redes sociales (publicación directa vía API de Instagram/Meta)

Alcance: Backend (OAuth2 + Webhooks)

Coste estimado: 30–40 h

ID: F-08

Mejora: IA para sugerencias de mejora (análisis de imagen con OpenAI Vision)

Alcance: Backend + IA

Coste estimado: 20–30 h

ID: F-09

Mejora: App móvil (PWA o React Native) para notificaciones y aprobación rápida

Alcance: Frontend móvil

Coste estimado: 60–80 h

ID: F-10

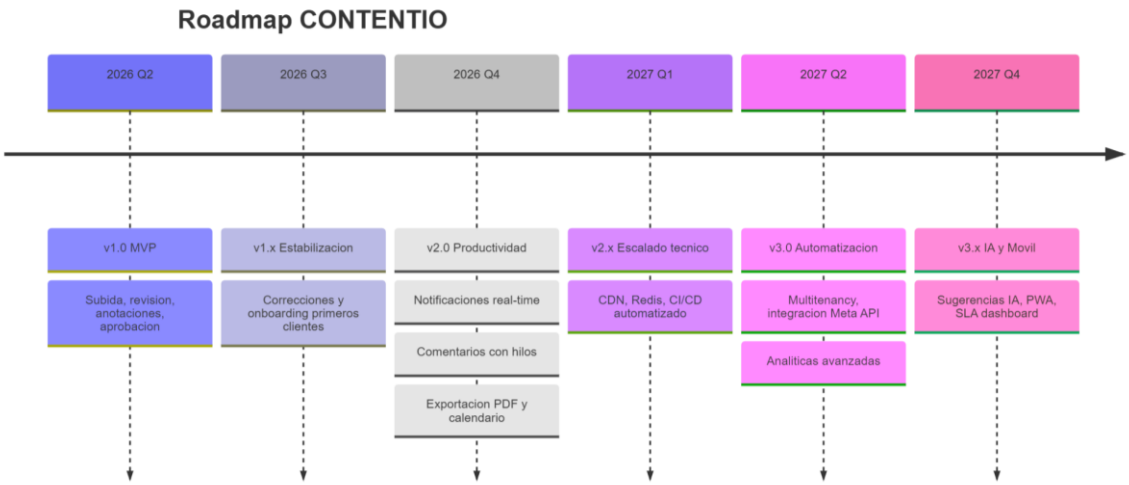
Mejora: Panel de analíticas (métricas de tiempos de aprobación, SLA por clínica)

Alcance: Backend + Frontend

Coste estimado: 20–28 h

Total v3.0 estimado: 170–238 horas de desarrollo adicional.

10.2 Roadmap de largo plazo



Tecnología:

- Migración de almacenamiento local a S3 (AWS o Cloudflare R2) al superar los 10 GB de activos.
- Incorporación de Redis para caché de sesiones y colas de trabajos (Laravel Queues).
- Pipeline CI/CD completo con GitHub Actions para despliegue automatizado al VPS.

Escalado:

- Separación del backend en microservicios si la carga supera los 500 usuarios concurrentes.
- Balanceo de carga con Nginx upstream o migración a un orquestador (Kubernetes) en ese escenario.

Mantenimiento:

- Actualizaciones de seguridad mensuales (dependabot + composer update / npm audit).
- Soporte LTS de Laravel 11 hasta 2027; migración a Laravel 12 planificada en v2.x.
- Copias de seguridad automáticas diarias de PostgreSQL con retención de 30 días.

11. Bibliografía y referencias

Las referencias se presentan en formato APA 7.ª edición, agrupadas por tipología. Al final del apartado se indica qué fuentes corresponden a cada sección del documento.

11.1 Fuentes académicas y metodológicas

Fowler, M. (2012, 1 de mayo). Test pyramid. martinfowler.com. https://martinfowler.com/bliki/TestPyramid.html

Moreno Martín, O. M. (2025). Tecnologías avanzadas de desarrollo: EB3.1 Modelado homogéneo de proyectos software mediante frameworks avanzados [Diapositivas de clase]. Universidad Pablo de Olavide.

Schwaber, K., & Sutherland, J. (2020). The Scrum guide: The definitive guide to Scrum: The rules of the game. Scrum.org. https://scrumguides.org/docs/scrumguide/v2020/2020-Scrum-Guide-US.pdf

11.2 Fuentes técnicas — Documentación oficial

Backend

Bergmann, S. (2024). PHPUnit 11 manual. PHPUnit. https://phpunit.de/documentation.html

Laravel LLC. (2024a). Laravel 11.x documentation. Laravel. https://laravel.com/docs/11.x

Laravel LLC. (2024b). Laravel queues. Laravel. https://laravel.com/docs/11.x/queues

Laravel LLC. (2024c). Laravel Reverb. Laravel. https://laravel.com/docs/11.x/reverb

Laravel LLC. (2024d). Laravel Sanctum. Laravel. https://laravel.com/docs/11.x/sanctum

Maduro, N. (2024). Pest PHP — an elegant PHP testing framework. Pest PHP. https://pestphp.com/docs/

The PHP Group. (2024). PHP 8.3 manual. PHP. https://www.php.net/manual/es/

The PostgreSQL Global Development Group. (2024). PostgreSQL 16 documentation. PostgreSQL. https://www.postgresql.org/docs/16/

Frontend

Axios Contributors. (2024). Axios documentation. Axios. https://axios-http.com/docs/intro

Microsoft. (2024a). Playwright documentation. Playwright. https://playwright.dev/docs/intro

Morote, E. S. M. (2024). Pinia — the intuitive store for Vue.js. Pinia. https://pinia.vuejs.org/

Tailwind Labs. (2024a). Tailwind CSS documentation. Tailwind CSS. https://tailwindcss.com/docs/

Vue.js. (2024a). Vue.js 3 — the progressive JavaScript framework. Vue.js. https://vuejs.org/guide/

Vue.js. (2024b). Vue Router 4 documentation. Vue Router. https://router.vuejs.org/

You, E. (2024a). Vite — next generation frontend tooling. Vite. https://vitejs.dev/guide/

Infraestructura y despliegue

Docker Inc. (2024). Docker documentation. Docker. https://docs.docker.com/

Electronic Frontier Foundation. (2024). Certbot documentation. Certbot. https://certbot.eff.org/docs/

F5 Inc. (2024). Nginx documentation. Nginx. https://nginx.org/en/docs/

Let's Encrypt. (2024). Let's Encrypt — free SSL/TLS certificates. Let's Encrypt. https://letsencrypt.org/docs/

OVH Group. (2024). OVHcloud — VPS solutions. OVHcloud. https://www.ovhcloud.com/es/vps/

Diseño

Google LLC. (2023). Material Design 3. Material Design. https://m3.material.io/

Inteligencia artificial y escalado (evolución futura)

OpenAI. (2024). GPT-4 Vision — OpenAI API documentation. OpenAI. https://platform.openai.com/docs/guides/vision

Redis Ltd. (2024). Redis documentation. Redis. https://redis.io/docs/

11.3 Repositorios y código fuente

Laravel LLC. (2024e). laravel/laravel [Software]. GitHub. https://github.com/laravel/laravel

Laravel LLC. (2024f). laravel/sanctum [Software]. GitHub. https://github.com/laravel/sanctum

Microsoft. (2024b). microsoft/playwright [Software]. GitHub. https://github.com/microsoft/playwright

Tailwind Labs. (2024b). tailwindlabs/tailwindcss [Software]. GitHub. https://github.com/tailwindlabs/tailwindcss

Vue.js. (2024c). vuejs/core [Software]. GitHub. https://github.com/vuejs/core

You, E. (2024b). vitejs/vite [Software]. GitHub. https://github.com/vitejs/vite

11.4 Referencias por apartado

Sección 4 — Estado del arte Análisis comparativo realizado mediante consulta directa a los sitios web oficiales de Bynder (https://www.bynder.com), Widen Collective (https://www.widen.com), Brandfolder (https://brandfolder.com) y Canto (https://www.canto.com).

Sección 5 — Requisitos Schwaber & Sutherland (2020) — metodología de historias de usuario y épicas.

Sección 6 — Propuesta de solución Laravel LLC (2024a); Vue.js (2024a); The PostgreSQL Global Development Group (2024); Docker Inc. (2024); Tailwind Labs (2024a); Laravel LLC (2024d) — Sanctum; Google LLC (2023) — Material Design 3; Moreno Martín (2025) — conceptos de Laravel y Composer.

Sección 7 — Plan de trabajo Schwaber & Sutherland (2020) — sprints y gestión ágil.

Sección 8 — Plan de pruebas Fowler (2012) — pirámide de pruebas; Bergmann (2024); Maduro (2024); Microsoft (2024a) — Playwright.

Sección 9 — Estrategia de despliegue Docker Inc. (2024); F5 Inc. (2024) — Nginx; Electronic Frontier Foundation (2024) — Certbot; Let's Encrypt (2024); OVH Group (2024); Moreno Martín (2025) — configuración de entorno con Composer.

Sección 10 — Evolución futura Laravel LLC (2024c) — Reverb; Redis Ltd. (2024); OpenAI (2024).